

AN INTERNSHIP REPORT

ON

ANA(AI Receptionist Assistant)

At Research Management Cell (RMC) Department, KCC

Submitted in partial fulfillment of the requirements of the degree of

Bachelor of Computer Applications

Submitted To:



Purbanchal University

Faculty of Science and Technology Biratnagar, Nepal

Submitted by:

Srijan Basnet

Exam Roll No: 370121

Registration No: 025-3-2-04314-2022

July, 2026

AN INTERNSHIP REPORT

ON

ANA(AI Receptionist Assistant)

At Research Management Cell (RMC) Department, KCC

Submitted in partial fulfillment of the requirements of the degree of

Bachelor of Computer Applications

Submitted To:

Purbanchal University

Faculty of Science and Technology Biratnagar, Nepal

Submitted by:

Srijan Basnet

Exam Roll No: 370121

Registration No: 025-3-2-04314-2022

Under the Supervision of:

Mr. John Subba

Supervisor

July, 2026

KCC LETTERHEAD

College Supervisor's Recommendation

Date: _____

This is to certify that the internship report titled "ANA(AI Receptionist)", carried out by Srijan Basnet under my supervision, is submitted in partial fulfillment of the academic requirements for the degree of Bachelor of Computer Applications of Purbanchal University.

I hereby recommend this report for final evaluation as it meets the necessary academic and technical standards set forth for the internship program.

Mr. John Subba

Supervisor

Kantipur City College

Putalisadak, Kathmandu

KCC LETTERHEAD

Letter of Approval

Date: _____

This is to certify that the internship report submitted by Srijan Basnet, entitled "ANA (AI Receptionist Assistant)", has been evaluated.

The report has been prepared in partial fulfillment of the requirements for the degree of Bachelor of Computer Applications. In our opinion, the report is satisfactory in both technical scope and academic quality, and meets the standards expected for the completion of the internship program.

Er. Rabi Shrestha

Internal Supervisor
Head, RMC

Kantipur City College

Mr. Saroj Pandey

Head, Department of IT
Kantipur City College

Mr. Raju Kattel

Principal

Kantipur City College

External Examiner

.....

.....

.....

Acknowledgement

I would like to express my sincere gratitude to Purbanchal University and Kantipur City College for providing me with the opportunity to complete this internship as a partial fulfillment of the academic requirements of my Bachelor of Computer Applications degree program.

I am deeply thankful to my internship supervisor, Mr. John Rai, for their continuous guidance, technical insights, and valuable feedback throughout the development of the speech processing systems. I would also like to thank the Head of the IT Department, Mr. Saroj Pandey, for coordinating the internship program and providing necessary institutional resources.

I extend my appreciation to the Research Management Cell (RMC) Department at Kantipur City College for offering a professional environment and necessary research infrastructure that allowed me to work on this AI/ML project. Collaborative work with my team members, who worked on retrieval indexing, backend routing, and talking-head rendering, was highly educational and motivating.

Finally, I would like to acknowledge the open-source community for the development of FAISS, Sentence-Transformers, and Qwen3 (served via Ollama/Lemonade), which made high-performance, local retrieval-augmented generation accessible on consumer hardware.

Abstract

This internship report presents the design, implementation, and optimization of Retrieval Augmented Generation(RAG) pipeline developed for the AI Receptionist Assistant (ANA) project at the Research Management Cell (RMC) Department of Kantipur City College. The main goal of the project was to implement an accurate, high-performance, and entirely local question-answering interface for college queries without depending on external cloud APIs.

During the internship period from May 13, 2026 to July 8, 2026, I was involved in local model benchmarking, retrieval tuning, and answer generation optimization. The RAG module utilizes FAISS for multi-file vector indexing combined with Sentence-Transformers embeddings, enabling efficient similarity search across college documents. To overcome domain-specific challenges, we implemented keyword-based routing, custom lexicon normalization for college acronyms (e.g. BCA-IT, BBA, BBS), and a hybrid answering strategy that combines retrieved context with generative responses. The generation module integrates Qwen3-1.7B served via an Ollama/Lemonade backend, producing natural, context-grounded answers directly on local hardware, supported by a keyword-based fallback interface for low-confidence queries.

The completed RAG pipeline was integrated into the local application server, achieving low-latency retrieval and response generation suitable for real-time interaction. The results demonstrate that local open-source RAG systems are highly suitable for interactive reception systems, preserving student privacy and avoiding recurring cloud API costs.

Table of Contents

Chapter 1: Introduction	1
1.1 Introduction.....	1
1.2 Problem Statement:	1
1.3 Objectives	2
1.4 Scope and Limitation	2
1.5 Report Organization	2
Chapter 2: Host Department.....	3
2.1 Host Details	3
2.2 Organizational Hierarchy	3
Chapter 3: System Research.....	4
3.1 System Overview	4
3.2 Technical Stack	4
3.3 Literature Review	5
Chapter 4: Roles and Implementation.....	6
4.1 Roles and Responsibilities	6
4.2 Implementation Details	6
4.3 TimeLine	7
4.4 Description of the Projects Involved.....	8
4.5 Tasks / Activities Performed	9
Chapter 5: Conclusion and Learning Outcomes	10
5.1 Conclusion.....	10
5.2 Learning Outcome.....	10
5.3 Future Enhancement.....	10

Chapter 1: Introduction

1.1 Introduction

An internship is a structured practical experience that allows students to apply academic knowledge to industry projects. This report covers the internship carried out as a partial requirement of the Bachelor of Computer Applications (BCA) program at Kantipur City College. The project, titled "AI Receptionist ANA", is an interactive, multi-modal receptionist system designed to assist students and visitors with institutional information at KCC. It integrates semantic search, voice transcription, natural language response generation, and talking-head rendering.

As an AI/ML Intern, my primary contribution was the design and implementation of the retrieval-augmented generation (RAG) interface. This interface consists of two major subsystems: a FAISS-based multi-file indexing pipeline to semantically search institutional documents, and a hybrid answering module that combines retrieved context with generative responses via a keyword routing mechanism. Both services were optimized for local execution to ensure low-latency responses, protecting student privacy and preventing ongoing cloud dependencies. The internship was carried out from May 13, 2026 to July 8, 2026 in the Research Management Cell (RMC) Department of Kantipur City College.

1.2 Problem Statement:

Retrieval and generation in local interactive reception desks face several challenges:

- Inconsistent document formatting, mixed file types, and unstructured institutional content often result in poor chunking and irrelevant retrieval in typical RAG systems.
- General-purpose embedding models perform poorly with domain acronyms (e.g. BCA-IT, BBA, BBS) and localized names (e.g. teacher and coordinator names like Ravi sir).
- Short or ambiguous queries (e.g., 'fees', 'timing', 'result') are easily misrouted without keyword-based routing, requiring custom matching thresholds.
- Large language model-based answer generation introduces computational latency, making the interactive agent feel sluggish and unresponsive during live conversations.
- Purely retrieval-based answering fails on queries with no direct document match, requiring a hybrid fallback strategy to avoid returning empty or irrelevant responses.

1.3 Objectives

The primary objectives of this internship were:

- To develop a local retrieval service using FAISS for multi-file semantic indexing of institutional documents.
- To build a hybrid answering system that combines retrieved context with generative responses using Qwen3 served via Ollama/Lemonade.
- To implement a robust keyword-based fallback mechanism to guarantee relevant responses in case of low-confidence retrieval.
- To customize retrieval and routing logic to prevent mismatched results for course names (BCA-IT) and staff names.
- To filter out low-confidence or irrelevant matches from the retrieved context before generation.

1.4 Scope and Limitation

The scope of my work covers the FAISS-based retrieval service, multi-file document indexing, keyword-based query routing, hybrid answer generation, lexicon normalization, and backend API service routes.

Limitations: The system uses embedding models optimized primarily for English. It does not natively support mixed English-Nepali code-switched queries or highly ambiguous, multiintent questions. It relies on local system hardware (a GPU is preferred for lowlatency generation via Ollama/Lemonade, falling back to CPU when unavailable), and retrieval accuracy depends on the quality and structure of the indexed source documents.

1.5 Report Organization

The report is organized as follows: Chapter 2 introduces the host department. Chapter 3 covers the background study and literature review of retrieval-augmented generation and semantic search techniques. Chapter 4 provides details on roles and implementation process. Chapter 5 concludes with outcomes and recommendations.

Chapter 2: Host Department

2.1 Host Details

The internship was conducted at the Research Management Cell (RMC) Department of Kantipur City College. The RMC is the dedicated research and development cell of the college. Its primary mission is to foster academic research, facilitate technological innovation, and execute software development projects that serve the academic and administrative requirements of the institution.

The department provides a collaborative ecosystem where researchers, IT faculty, and software development teams also work on digital solutions, machine learning projects, and natural language interfaces.

2.2 Organizational Hierarchy

The department is structured as follows to ensure collaborative execution of AI/ML and software engineering projects:

- I. Director / Head of Research: Overlooks strategic planning, policy, and research funding.
- II. Project Coordinator: Manages project schedules, assigns developer tasks, and monitors delivery.
- III. AI/ML Research Supervisor: Guides neural model selection, training, quantization, and pipeline design.
- IV. AI/ML Interns (Speech & NLP): Handles local model wrapping, parameter tuning, and benchmarking.
- V. Software Engineering Interns: Integrates model endpoints with user interfaces and backend database services.

Chapter 3: System Research

3.1 System Overview

The RAG module serves as the brain of the AI receptionist. When a user asks a question, the system follows this pipeline:

- Step 1: User input is received as a text query
- Step 2: A keyword classifier checks if the query is college-related
- Step 3: If college-related, the query is embedded and searched across all FAISS indexes
- Step 4: The most relevant document chunks are retrieved
- Step 5: Retrieved chunks are passed to the LLM along with the query and chat history
- Step 6: The LLM generates a natural language response
- Step 7: If no relevant chunks are found, the LLM answers from general knowledge

3.2 Technical Stack

Language and Libraries

1. Python 3.11
2. sentence-transformers (all-MiniLM-L6-v2) for semantic embeddings
3. FAISS (faiss-cpu) for vector similarity search
4. LangChain text splitters for document chunking
5. PyMuPDF (fitz) for PDF text extraction
6. OpenAI Python client for Lemonade server communication

LLM and Server

1. Lemonade Server (local LLM hosting, port 13305)
2. Qwen3-1.7B-GGUF model (lightweight, runs locally)
3. Fully offline — no internet required after setup

3.3 Literature Review

A review of literature on conversational AI systems shows a clear shift toward local model execution in institutional settings. While cloud-based APIs (such as OpenAI's GPT models or commercial vector database services) offer high accuracy, they suffer from internet latency, recurring monthly costs, and data privacy issues. Grounding a college assistant locally on a single workstation requires lightweight, efficient architectures like FAISS for vector similarity search and compact instruction-tuned language models such as Qwen3 for generation. Modern systems mitigate accuracy limitations in domain-specific settings by using keyword-based routing, custom lexicon normalization, and hybrid retrieval-generation strategies, matching cloud performance locally.

The quality of retrieval in turn depends heavily on the embedding model used to represent text; Sentence-Transformers, based on the Siamese BERT-network architecture proposed by Reimers and Gurevych (2019), has become a widely adopted standard for producing semantically meaningful sentence embeddings suited to similarity search tasks. Modern systems mitigate accuracy limitations in domain-specific settings by using keyword-based routing, custom lexicon normalization, and hybrid retrieval-generation strategies. This hybrid approach, combining sparse keyword matching with dense semantic retrieval, has been shown in recent literature to outperform either method alone, particularly for short or ambiguous queries common in conversational interfaces, allowing local deployments to closely match the performance of cloud-hosted systems while preserving data privacy and eliminating recurring API costs.

Chapter 4: Roles and Implementation

4.1 Roles and Responsibilities

My responsibility in the ANN project was to design, build, and integrate the RAG module. Specifically, I was responsible for:

- Building a multi-file document indexing system using FAISS
- Implementing semantic search using sentence-transformers embeddings
- Developing a query router to identify the most relevant document index
- Integrating the LLM (via Lemonade server) for answer generation
- Implementing a hybrid answering system (document-based and general knowledge)

4.2 Implementation Details

- I. **Document Indexing:** Each knowledge file (TXT or PDF) is processed separately. Text is extracted, split into 500 character chunks with 50-character overlap, embedded using allMiniLM-L6-v2, and stored in a dedicated FAISS index. This allows precise retrieval from the correct source file.
- II. **Query Routing:** A keyword-based classifier first checks if the query contains institution related terms (e.g., KCC, admission, BCA, semester, faculty). If matched, the system searches across all FAISS indexes and selects the one with the lowest distance score. If no relevant chunks are found within the relevance threshold, the query falls back to general knowledge.
- III. **Answer Generation:** The retrieved chunks are injected into a structured prompt along with the full chat history, enabling the LLM to generate contextually accurate and conversational responses. Streaming is enabled for real-time token-by-token output. A postprocessing step removes markdown symbols to ensure clean plain-text output.
- IV. **Chat History:** Every user query and assistant response is stored in a chat history list. This full history is sent with every API call to the LLM, enabling multi-turn conversations where the assistant remembers previous context.

4.3 TimeLine

Duration	Activities Performed	Technical Details
Week 1	Orientation & Setup	Familiarized with RMC department workflows; configured local Python environment, installed FAISS, Sentence-Transformers, and Ollama on Windows.
Week 2	Architecture Analysis	Studied existing ANA repository scaffold; benchmarked embedding models for retrieval accuracy and speed.
Week 3	Document Preprocessing	Designed chunking strategy for institutional documents; implemented source tagging for traceability.
Week 4	Indexing Pipeline	Wrote initial FAISS indexing script; implemented multi-file loading and vector store persistence.
Week 5	Keyword Routing	Implemented keyword based routing layer for domain acronyms (KCC, BCA-IT, BBA) and staff names.
Week 6	LLM Backend Setup	Wrote hybrid answering logic combining retrieved context with generated responses; designed prompt templates.

Week 7	Hybrid Answering	Wrote hybrid answering logic combining retrieved context with generated responses; designed prompt templates.
Week 8	Confidence Thresholding	Implemented similarity score thresholds to trigger
Week 9	FastApi Integration	keyword fallback on low confidence retrieval.
Week 10	Query Refinement	Tuned chunk size and overlap parameters; filtered irrelevant matches from retrieved context
Week 11	Testing & Edge Cases	Tested ambiguous and short queries; refined routing logic.
Week 12	Report & Documentation	Compiled retrieval accuracy notes, analyzed performance bottlenecks, and finalized the internship report.

4.4 Description of the Projects Involved

The project I worked on is ANA, an AI Receptionist assistant built under the Research Management Cell (RMC) of Kantipur City College. The assistant accepts user speech, transcribes it locally, queries a FAISS vector index of college policies and admission criteria, passes the context to a local Qwen language model, and returns both text, synthesized audio, and a lip-synced video avatar response.

4.5 Tasks / Activities Performed

The implementation details of the two core services I developed are:

- I. Retrieval service: The service wraps FAISS for vector similarity search across multiple institutional documents. We configured a multi-file indexing pipeline using Sentence-Transformers embeddings, with each document chunked and tagged by source for traceability. We also designed a keyword routing layer: a query is matched against a domain-specific keyword map first, and falls back to dense vector search when no strong keyword match is found, ensuring acronyms like BCA-IT are routed correctly instead of being lost in generic semantic similarity.
- II. Answer generation wrapper: The service uses Qwen3-1.7B served through a local Ollama/Lemonade backend. It combines the top retrieved chunks into a context window, constructs a grounded prompt template, and streams the generated response back through the API layer. A confidence threshold on retrieval similarity scores determines whether the system answers directly from retrieved context or falls back to a general keyword-based response, preventing irrelevant or hallucinated answers when no matching document exists.

Chapter 5: Conclusion and Learning Outcomes

5.1 Conclusion

During this internship, I successfully designed and implemented the RAG module for the ANA AI Receptionist project. The module supports multi-file document retrieval, hybrid answering (document + general knowledge), offline LLM inference via Lemonade server, and multi-turn chat memory. The system is fully functional and integrates with the broader ANN project pipeline. This internship gave me hands-on experience with RAG architecture, vector databases, local LLM deployment, and building production-ready AI systems from scratch.

5.2 Learning Outcome

Key learning outcomes from the internship include:

- Practical experience in vector indexing, embedding generation, and similarity search optimization using FAISS.
- Deep understanding of chunking strategies, keyword-based routing, and multi-file document handling.
- Knowledge of local LLM inference (Qwen3) served via Ollama/Lemonade and hybrid answering strategies.
- Debugging indexing issues, query mismatches, and implementing confidence-based fallback logic.

5.3 Future Enhancement

Future recommendations include exploring reranking models to improve retrieval precision on ambiguous queries, integrating a feedback loop where flagged low-confidence answers are used to expand the document index, and experimenting with query expansion techniques to better handle short or vague student questions.

References

- I. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. FAISS. Retrieved July 8, 2026, from <https://github.com/facebookresearch/faiss>
- II. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Sentence-Transformers. Retrieved July 8, 2026, from <https://www.sbert.net/>
- III. Ollama(n.d.). Ollama documentation. Retrieved July 8, 2026, from <https://ollama.com/>
- IV. Qwen Team.(2024). Qwen3 technical report. Retrieved July 8, 2026, from <https://github.com/QwenLM/Qwen3>